WORKSHEET PRAKTIKUM 3

M.Fahmi Hadi Wijaya

A. Class

Class	State yang Dibutuhkan	Behavior yang Dibutuhkan
Mahasiswa	nim, nama	menyediakan identitas pemesan
MenuItem	kode, nama, harga, tersedia	memberi info harga/ketersediaan; mengubah ketersediaan lewat behavior (tandaiHabis)
Pesanan	nomor, pemesan (reference), item (reference), jumlah	memeriksa dapatDiproses(), menghitung hitungTotal(), menampilkan ringkasan()

B. Tahap

1. Bangun Class Mahasiswa

Pertanyaan / Prediksi	Jawaban
Apakah m1 dan m2 menggunakan class yang sama?	Ya. Keduanya dibuat dari class Mahasiswa yang sama menggunakan operator new. Class hanya satu (cetakan/blueprint), tetapi object yang dihasilkan bisa lebih dari satu.
Mengapa nilai state-nya bisa berbeda?	Karena setiap object punya area memori sendiri untuk menyimpan field-nya. Nilai nim dan nama diisi lewat parameter constructor saat masing-masing object dibuat, sehingga m1 dan m2 menyimpan data yang berbeda meskipun berasal dari class yang identik.
Apa yang terjadi jika menulis System.out.println(m1.nama); dari Main?	Program gagal dikompilasi (compile error) karena field nama bersifat private. Field private hanya bisa diakses dari dalam class Mahasiswa itu sendiri, bukan dari class lain seperti Main. Compiler akan menampilkan pesan semacam 'nama has private access in Mahasiswa'. Akses yang benar harus lewat getter: m1.getNama().

2. MenuItem dan Encapsulation

   Ketika baris kopi.tersedia = false; ditulis langsung di class Main, compiler menolak dengan error kurang lebih: 'tersedia has private access in MenuItem'. Konsep yang ditegakkan: encapsulation. Field tersedia disembunyikan (private) agar tidak bisa diubah sembarangan dari luar object. Perubahan status hanya boleh dilakukan lewat behavior yang sudah disediakan object itu sendiri, yaitu method tandaiHabis(). Dengan begitu, object MenuItem yang mengendalikan aturan perubahan state-nya sendiri, bukan kode luar yang menebak-nebak cara mengubahnya.

3. Pesanan, Relationship, dan Kolaborasi Object

Observasi	Penjelasan

Mengapa p1 dan p2 dapat memiliki pemesan berbeda?	Karena field pemesan pada Pesanan bertipe Mahasiswa (reference), bukan tipe data primitif yang disalin. Saat p1 dibuat, ia menerima reference ke object mhs1; saat p2 dibuat, ia menerima reference ke object mhs2 yang berbeda. Setiap object Pesanan bebas menunjuk ke object Mahasiswa manapun yang diberikan lewat constructor.
Mengapa p2 dapat ditolak setelah kopi ditandai habis?	Karena dapatDiproses() memanggil item.isTersedia() untuk bertanya langsung ke object MenuItem yang direferensikan. Begitu kopi.tandaiHabis() dipanggil, state tersedia pada object kopi berubah menjadi false. Karena p2 hanya menyimpan reference ke object kopi (bukan salinan datanya), perubahan itu langsung terlihat saat p2.dapatDiproses() dievaluasi.
Object mana yang mengetahui harga? Mengapa bukan Pesanan yang menyimpan copy harga?	Object MenuItem yang mengetahui dan menyimpan harga. Pesanan sengaja tidak menyimpan salinan harga, karena jika Pesanan menyimpan copy dan harga menu berubah di kemudian hari, salinan pada Pesanan akan menjadi data basi (tidak sinkron). Dengan menyimpan reference dan memanggil item.getHarga(), Pesanan selalu mendapat harga yang paling mutakhir.
Di baris mana message antar-object terjadi?	Pada method hitungTotal() (memanggil item.getHarga()) dan dapatDiproses() (memanggil item.isTersedia()). Kedua baris ini adalah contoh Pesanan mengirim message/method call kepada object MenuItem untuk meminta informasi, tanpa perlu tahu bagaimana MenuItem menyimpan data tersebut secara internal.

4. Static vs Instance

Prediksi sebelum run: nomor p1 = 1, p2 = 2, p3 = 3 (meningkat sesuai urutan pembuatan object).
Hasil setelah dijalankan: sesuai prediksi — nomor bertambah 1, 2, 3 karena setiap constructor memanggil nextNumber++.

Tabel hasil eksperimen static sebelum/sesudah

Kondisi nextNumber	**Hasil nomor p1, p2, p3**	**Kesimpulan**
static (kode asli)	1, 2, 3	nextNumber dibagi bersama oleh seluruh object Pesanan, sehingga nilainya terus meningkat setiap kali constructor dipanggil, tidak peduli object mana yang membuatnya.
tanpa static (field instance biasa)	1, 1, 1	Setiap object mendapat salinan nextNumber miliknya sendiri yang selalu mulai dari 1, sehingga nomor tidak pernah meningkat antar object.

Kesimpulan: nomor adalah state per-object (unik untuk setiap Pesanan), sedangkan nextNumber adalah state shared milik class Pesanan secara keseluruhan. Pemilihan static harus mengikuti kepemilikan data — data yang secara konsep 'dimiliki bersama oleh semua instance' yang layak dijadikan static, bukan sekadar alasan kepraktisan.

5. Package

   Class Mahasiswa, MenuItem, dan Pesanan dipindahkan ke package model, sedangkan Main dipindahkan ke package app dan meng-import ketiga class model tersebut. Setelah struktur folder fisik (model/ dan app/) dibuat sesuai dengan deklarasi package pada tiap file, program berhasil dikompilasi dan dijalankan tanpa error.

C. Eksperimen

1. private vs public

Pertanyaan	**Jawaban**
Apa yang berubah ketika private menjadi public?	Ketika tersedia diubah menjadi public, baris menu.tersedia = false; yang tadinya ditolak compiler menjadi bisa dikompilasi dan dijalankan tanpa error, karena field bisa diakses langsung dari class manapun.
Apakah public membuat program lebih fleksibel? Risiko desain apa yang muncul?	Secara teknis lebih 'mudah' diakses, tapi ini bukan fleksibilitas yang sehat. Risikonya: aturan bisnis bisa dilanggar begitu saja dari luar (misalnya field diubah ke nilai yang tidak valid tanpa validasi apapun), state object jadi tidak terjamin konsistensinya, dan jika suatu saat aturan perubahan perlu ditambah (misalnya harus dicatat log setiap kali status berubah), perubahan itu harus dicari di banyak tempat kode yang mengakses field secara langsung, bukan cukup di satu method.
Mengapa tandaiHabis() lebih bermakna daripada akses langsung ke field?	Karena tandaiHabis() adalah behavior yang jelas menyatakan maksud ('tandai menu ini sebagai habis'), sementara menu.tersedia = false; hanya operasi teknis tanpa konteks. Dengan method, logika perubahan state terpusat di satu tempat (di dalam class MenuItem), sehingga lebih mudah dijaga konsistensinya dan lebih mudah dibaca oleh siapapun yang memakai class tersebut.

2. Constructor vs Object Setengah Jadi

   Ketika Mahasiswa hanya memiliki constructor berparameter (String nim, String nama) dan dipanggil new Mahasiswa(); tanpa argumen, compiler menolak dengan error kurang lebih: 'constructor Mahasiswa in class Mahasiswa cannot be applied to given types; required: String, String — found: no arguments'.

   Jika constructor kosong ditambahkan sementara, object bisa dibuat tanpa nim dan nama (nilainya menjadi null). Object seperti ini tidak masuk akal untuk kasus kantin kampus, karena setiap Mahasiswa yang memesan wajib memiliki identitas (NIM dan nama) — tanpa data itu, object tidak bisa digunakan secara berguna oleh Pesanan maupun ditampilkan di ringkasan().

   Makna teknis: constructor bukan sekadar method khusus, tetapi kontrak yang menentukan data minimal apa saja yang wajib dipenuhi agar sebuah object 'boleh lahir' dan dianggap valid dalam program.

3. Static vs Instance

Sudah dibuktikan pada Tahap 5: dengan nextNumber static, nomor tiga Pesanan berturut-turut adalah 1, 2, 3. Setelah static dihapus dan nextNumber dijadikan field instance (reset ke 1 di setiap object), hasilnya menjadi 1, 1, 1 untuk ketiga object, karena masing-masing object punya salinan nextNumber sendiri yang tidak saling memengaruhi. Perbedaan ini membuktikan bahwa static field adalah milik class (dibagi bersama), sedangkan field instance biasa adalah milik masing-masing object secara terpisah.

4. Relationship: Reference vs Duplikasi Data

Pertanyaan	**Jawaban**
Desain mana yang menunjukkan relationship antar-object secara langsung?	Desain A (menyimpan private Mahasiswa pemesan; dan private MenuItem menu;). Desain ini secara eksplisit menyatakan bahwa Pesanan 'memiliki/mengenal' object Mahasiswa dan MenuItem lewat reference, sesuai konsep has-a dalam OOP.
Jika nama mahasiswa berubah, desain mana yang berpotensi menyimpan copy lama?	Desain B. Karena Desain B menyalin nilai (namaPemesan, namaMenu, dsb) ke dalam field Pesanan sendiri, jika data asli di object Mahasiswa/MenuItem berubah setelah Pesanan dibuat, salinan pada Pesanan tidak ikut ter-update dan menjadi data basi (stale data). Desain A tidak mengalami masalah ini karena selalu membaca data terbaru lewat reference.
Apakah menyimpan object reference selalu otomatis benar?	Tidak selalu. Reference cocok dipakai ketika data memang harus selalu sinkron dengan sumber aslinya dan object yang direferensikan tetap ada/valid selama Pesanan digunakan. Namun ada kasus lain di mana menyimpan salinan data justru lebih tepat, misalnya untuk mencatat riwayat/histori (harga pada saat transaksi terjadi, meskipun harga menu berubah di kemudian hari) — di situ snapshot data justru dibutuhkan agar catatan transaksi lama tidak berubah-ubah mengikuti data terbaru.

5. Package dan Compiler

Ketika salah satu baris import (misalnya import model.MenuItem;) dihapus dari Main.java, compiler menolak dengan error 'cannot find symbol' pada setiap baris yang memakai class MenuItem, karena Main tidak lagi mengenal class tersebut tanpa import yang sesuai.

Ketika package declaration pada salah satu file (misalnya Pesanan.java) diubah tidak sesuai lokasi foldernya, IDE/compiler menampilkan error 'The declared package ... does not match the expected package', persis seperti error yang sempat muncul saat proses menyusun package pada praktikum ini. Ini membuktikan bahwa Java mewajibkan kecocokan ketat antara struktur folder fisik dan deklarasi package pada source code.

D. Tugas Praktikum Terintegrasi

1. Lengkapi Model Teknis

Ketiga class (Mahasiswa, MenuItem, Pesanan) sudah memiliki constructor, getter yang diperlukan, dan behavior yang relevan (isTersedia(), tandaiHabis(), dapatDiproses(), hitungTotal(), getStatus(), ringkasan(), getJumlahPesananDibuat()). Tidak dibuat setter untuk semua field secara otomatis — field tersedia misalnya hanya bisa diubah lewat tandaiHabis(), bukan lewat setter umum, karena perubahan status habis punya makna

bisnis tersendiri, bukan sekadar 'ganti nilai'. Semua field domain (nim, nama, kode, nama, harga, tersedia, nomor, pemesan, item, jumlah) bersifat private.

2. Buktikan Class dan Object

   Dibuat 2 object Mahasiswa (mhs1 = "241001"/"Asep", mhs2 = "241002"/"Siti"), 2 object MenuItem (menu1 = Nasi Goreng, menu2 = Kopi Susu), dan 3 object Pesanan (p1, p2, p3) dengan kombinasi pemesan dan menu yang berbeda-beda.

   Mahasiswa, MenuItem, dan Pesanan adalah class (cetakan/blueprint yang mendefinisikan struktur data dan perilaku). mhs1, mhs2, menu1, menu2, p1, p2, p3 adalah object/instance — wujud konkret hasil operator new dari class tersebut, masing-masing menyimpan state sendiri-sendiri meskipun berasal dari class yang sama.

3. Buktikan Encapsulation dan Constructor

   Bukti error compiler untuk encapsulation: percobaan menulis menu1.tersedia = false; langsung dari Main menghasilkan compile error 'tersedia has private access in MenuItem' (lihat Eksperimen 1).

   Bukti object tidak dapat dibuat dengan constructor yang tidak tersedia: percobaan new Mahasiswa(); tanpa argumen menghasilkan compile error karena tidak ada constructor kosong yang cocok pada class Mahasiswa (lihat Eksperimen 2).

   Constructor yang dipilih — Mahasiswa(String nim, String nama), MenuItem(String kode, String nama, int harga), dan Pesanan(Mahasiswa pemesan, MenuItem item, int jumlah) — sesuai untuk state awal object karena setiap object memang membutuhkan data-data tersebut sejak awal dibuat agar bisa berfungsi dengan benar. Tanpa NIM/nama, Mahasiswa tidak bisa diidentifikasi; tanpa kode/nama/harga, MenuItem tidak bisa dijual; tanpa pemesan/item/jumlah, Pesanan tidak punya arti.

4. Buktikan Static

   Static counter nextNumber digunakan pada class Pesanan untuk menghasilkan nomor urut otomatis. Nomor tiga Pesanan (p1, p2, p3) tercetak 1, 2, 3 sesuai urutan pembuatan.

   Counter (nextNumber) bersifat static karena nilainya harus dibagi dan diakumulasi bersama oleh seluruh object Pesanan yang pernah dibuat — ini adalah data milik class secara keseluruhan. Sebaliknya, nomor tidak static karena setiap Pesanan harus memiliki nomor uniknya sendiri yang tidak berubah meskipun ada Pesanan lain dibuat setelahnya — ini adalah data milik masing-masing object.

   Method static getJumlahPesananDibuat() ditambahkan untuk mengembalikan jumlah total object Pesanan yang sudah diberi nomor (nextNumber - 1), dan dipanggil lewat nama class: Pesanan.getJumlahPesananDibuat().

5. Buktikan Relationship

   Pesanan menyimpan reference ke Mahasiswa (field pemesan) dan MenuItem (field item), bukan menyalin data mentahnya. dapatDiproses() menggunakan item.isTersedia() (method call, bukan akses field langsung), dan hitungTotal() menggunakan item.getHarga() * jumlah.

6. Package Final

Mahasiswa.java, MenuItem.java, dan Pesanan.java berada pada package model; Main.java berada pada package app dan meng-import ketiga class model lewat import model.Mahasiswa;, import model.MenuItem;, dan import model.Pesanan;. Struktur folder fisik (model/ dan app/) sudah dibuat cocok dengan deklarasi package pada masing-masing file, sehingga program dapat dikompilasi dan dijalankan tanpa error package/import.

E. Independent Challenge

1. Relationship uses-a dengan Kasir

Class Kasir tidak menyimpan Pesanan sebagai field, melainkan hanya menerima Pesanan sebagai parameter method proses(Pesanan pesanan). Ini adalah contoh relationship uses-a: Kasir 'meminjam' object Pesanan hanya selama method proses() berjalan, lalu tidak menyimpan referensinya sama sekali setelah method selesai.

Perbedaan dengan has-a pada Pesanan → Mahasiswa/MenuItem: pada has-a, Pesanan menyimpan reference sebagai field (private Mahasiswa pemesan;) yang bertahan selama object Pesanan itu masih ada/hidup — relationship-nya bersifat jangka panjang dan menjadi bagian dari state object. Pada uses-a, Kasir hanya memakai Pesanan sesaat lewat parameter method, tanpa relationship itu tercatat sebagai bagian permanen dari state Kasir.

2. Pertanyaan Menuju Inheritance

Jika kelak ada MenuMakanan dan MenuMinuman, keduanya bisa dikatakan is-a MenuItem, karena keduanya secara konsep adalah 'jenis MenuItem yang lebih khusus' — sama-sama memiliki kode, nama, harga, dan status tersedia, tetapi mungkin punya atribut tambahan yang spesifik (misalnya MenuMakanan punya level pedas, MenuMinuman punya ukuran/suhu). Hubungan is-a berarti MenuMakanan dan MenuMinuman dapat diperlakukan sebagai MenuItem pada konteks umum (misalnya saat dipakai di dalam Pesanan), namun tetap punya karakteristik khusus masing-masing. Ini belum diimplementasikan pada modul ini karena termasuk konsep inheritance yang akan dibahas pada pertemuan berikutnya.

F. Lesson Learn

Pertanyaan Refleksi	**Jawaban**
Baris kode mana yang paling membantu memahami perbedaan class dan object?	Baris Mahasiswa m1 = new Mahasiswa("241001", "Asep"); dan Mahasiswa m2 = new Mahasiswa("241002", "Siti");. Keduanya menggunakan class yang persis sama, tetapi menghasilkan dua object dengan data yang berbeda — ini membuat perbedaan 'cetakan' (class) dan 'hasil cetakan' (object) terasa nyata, bukan sekadar definisi di teori.
Apa perbedaan antara 'field private' dan 'object benar-benar menjaga state-nya'?	Field private hanya mencegah akses langsung dari luar, tapi itu belum tentu berarti state-nya terjaga dengan baik. Object benar-benar menjaga state-nya ketika perubahan hanya bisa terjadi lewat behavior yang punya aturan jelas (contohnya tandaiHabis() yang hanya bisa mengubah tersedia menjadi false, bukan setter bebas yang menerima nilai apa saja).

Apa yang terjadi ketika constructor kosong diizinkan? Apakah object masih masuk akal?	Object bisa dibuat tanpa data penting (nim dan nama menjadi null), tetapi object seperti ini tidak masuk akal untuk kasus kantin kampus karena identitas mahasiswa adalah data wajib yang dibutuhkan sejak awal, bukan data opsional.
Mengapa nextNumber cocok static tetapi nomor tidak?	nextNumber adalah 'milik bersama' seluruh class Pesanan (satu penghitung global), sedangkan nomor adalah 'milik masing-masing' object Pesanan (nilai unik yang tidak berubah setelah object dibuat).
Bagaimana relationship antar-class terlihat secara nyata di source code?	Lewat field bertipe class lain (private Mahasiswa pemesan; dan private MenuItem item;) serta lewat method call antar-object seperti item.getHarga() dan item.isTersedia() di dalam class Pesanan.
Apa fungsi package selain membuat folder terlihat rapi?	Package memastikan nama class tidak bentrok antar bagian program yang berbeda (namespace), mengelompokkan class berdasarkan tanggung jawabnya (model vs app), dan compiler memvalidasi kecocokan struktur folder dengan deklarasi package sehingga struktur project tetap konsisten dan mudah ditelusuri.
Satu konsep apa yang masih paling membingungkan setelah praktikum?	(Diisi sesuai pengalaman pribadi masing-masing mahasiswa saat mengerjakan praktikum.)